RideShare Data Engineering Project

Final Internship Project Report

Project Title: Ride-Sharing Analytics and Driver Performance Pipeline
Technology Stack: Python, PySpark, Apache Spark, Spark SQL, Databricks, Parquet
Architecture: Medallion Architecture
Project Type: Data Engineering Capstone Project

---

1. Introduction

Ride-sharing platforms generate large amounts of operational data related to drivers, trips, locations, fares, cancellations, and delays. However, raw data stored across multiple files cannot directly provide reliable business insights.

This project develops an end-to-end data engineering pipeline to ingest, clean, integrate, validate, and analyze ride-sharing data using Apache Spark and PySpark on Databricks.

The pipeline transforms raw datasets into structured analytical outputs that can be used to evaluate driver performance, analyze cancellation and delay patterns, identify high-demand pickup locations, and understand revenue distribution.

The project follows the Medallion Architecture consisting of Bronze, Silver, and Gold data layers.

2. Problem Statement

The ride-sharing datasets contain driver details, trip information, and operational trip logs in separate CSV files.

Without a structured processing pipeline, it is difficult to answer important business questions such as:

* Which drivers are performing the best?
* What is the cancellation rate for each driver?
* Which pickup locations have the highest demand?
* Which cities generate the most revenue?
* Which drivers experience the highest average delay?
* How many trips are completed or cancelled?
* Are there duplicate, missing, or inconsistent records?

The objective of this project is to build a scalable Spark-based pipeline that transforms raw operational data into clean, validated, and business-ready analytical datasets.

3. Project Objectives

The main objectives of this project are:

* Ingest raw CSV datasets using PySpark DataFrames
* Preserve raw data without transformation
* Apply explicit schemas to all datasets
* Perform data profiling and validation
* Remove duplicate and invalid records
* Handle null values using business rules
* Join drivers, trips, and trip log datasets
* Create derived columns for analytics
* Generate driver performance metrics
* Calculate cancellation and delay rates
* Identify high-demand pickup locations
* Analyze city-wise revenue
* Rank drivers using Spark window functions
* Validate data consistency across all stages
* Store processed and analytical datasets in Parquet format
* Export final business-ready outputs in CSV format

4. Dataset Description

4.1 Drivers Dataset

The drivers dataset contains the following fields:

Column	Description
driver_id	Unique identifier for each driver
name	Driver name
city	Driver operating city
rating	Driver rating

4.2 Trips Dataset

The trips dataset contains the following fields:

Column	Description
trip_id	Unique identifier for each trip
driver_id	Driver associated with the trip
pickup_location	Trip pickup location
drop_location	Trip drop location
distance_km	Trip distance in kilometres
fare_amount	Trip fare amount
trip_status	Completed or Cancelled

4.3 Trip Logs Dataset

The trip logs dataset contains the following fields:

Column	Description
log_id	Unique identifier for the log
trip_id	Related trip identifier

start_time	Trip start timestamp
end_time	Trip end timestamp
delay_minutes	Delay in minutes
cancellation_flag	Cancellation indicator

5. Architecture and Data Flow

The project follows the Medallion Architecture.

Bronze Layer

The Bronze Layer stores the source datasets in raw form.

Main activities:

* Read CSV files using PySpark
* Apply explicit schemas
* Preserve source data without business transformations
* Validate source record counts
* Store raw datasets in Parquet format

Silver Layer

The Silver Layer creates a clean, integrated, and enriched trip-level dataset.

Main activities:

* Remove duplicate records
* Validate null values
* Validate ratings, fares, distances, and delays
* Validate trip status and cancellation flags
* Join drivers, trips, and trip logs
* Create derived analytical columns
* Store cleaned data in Parquet format

Gold Layer

The Gold Layer generates business-ready analytical tables.

Main outputs:

* Driver performance
* Cancellation analysis
* Revenue analysis
* Delay analysis
* High-demand pickup locations
* Business KPI summary
* Overall driver ranking
* City-wise driver ranking

Data Flow

```text
Raw CSV Files
      ↓
Bronze Layer
      ↓
Silver Layer
      ↓
Gold Layer
      ↓
CSV Analytical Outputs
```

6. Technology Stack

The following technologies were used:

* **Python:** Main programming language
* **PySpark:** Distributed data processing
* **Apache Spark:** Processing engine
* **Spark SQL:** SQL-based analytics
* **Databricks:** Development and execution platform
* **Parquet:** Storage format for processed datasets
* **Git and GitHub:** Version control and project submission
* **CSV:** Final export format

7. Bronze Layer Implementation

The Bronze Layer was implemented using explicit PySpark schemas.

The following datasets were ingested:

* `drivers.csv`
* `trips.csv`
* `trip_logs.csv`

Each dataset contained 150 records.

After ingestion:

* Drivers records: 150
* Trips records: 150
* Trip log records: 150

All source records were preserved, and no records were lost during ingestion.

The datasets were written to the Bronze Layer in Parquet format.

8. Silver Layer Implementation

The Silver Layer transformed the raw Bronze datasets into a clean and integrated trip-level dataset.

Data Quality Checks

The following validations were performed:

* Duplicate driver IDs
* Duplicate trip IDs
* Duplicate log IDs
* Null values
* Driver rating range
* Negative trip distance
* Negative fare amount
* Negative delay values
* Trip status validity
* Cancellation flag validity
* Completed-trip timestamp consistency

No duplicate primary keys were found.

The only null values were found in the `end_time` column for cancelled trips. These null values were treated as valid business conditions because cancelled trips do not have a completed end timestamp.

Dataset Integration

The following joins were performed:

```text
Trips.driver_id = Drivers.driver_id
Trips.trip_id = Trip_Logs.trip_id
```

The final joined dataset contained 150 records, which confirmed that no trip records were lost during integration.

Derived Columns

The following columns were created:

* `trip_duration_minutes`
* `completion_flag`
* `is_delayed`
* `revenue_per_km`
* `delay_category`
* `trip_date`
* `trip_hour`

The final Silver dataset was stored in Parquet format.

9. Gold Layer Implementation

The Gold Layer generated aggregated analytical datasets for business reporting.

Driver Performance Metrics

The following metrics were calculated:

* Total trips
* Completed trips
* Cancelled trips
* Completion rate
* Cancellation rate
* Average delay
* Total revenue
* Average fare
* Average trip duration

Driver Ranking

Spark window functions were used to generate:

* Overall driver ranking
* City-wise driver ranking

Drivers were ranked using:

* Completion rate
* Total revenue
* Average delay
* Driver rating

Business Analytics

The Gold Layer generated:

* Driver performance analysis
* Cancellation analysis
* Revenue analysis
* Delay analysis
* High-demand pickup-location analysis
* Overall business KPI summary

The Gold outputs included:

* 99 active drivers
* 5 cities
* 5 pickup locations
* 1 overall KPI summary row

10. Spark SQL Analysis

The Silver dataset was registered as a temporary Spark SQL view named `silver_ride_data`.

Spark SQL was used to perform:

* Driver performance analysis

* Cancellation rate analysis
* Revenue analysis
* Delay analysis
* Overall driver ranking
* City-wise driver ranking

This project demonstrates the combined use of PySpark DataFrame APIs and Spark SQL within the same data engineering pipeline.

11. Data Validation

The following validation checks were implemented:

* Raw-to-Bronze record count reconciliation
* Duplicate primary-key validation
* Referential integrity validation
* Missing driver-reference validation
* Missing trip-log-reference validation
* Completed-trip timestamp validation
* Trip status and cancellation-flag consistency
* Numeric-value validation
* Silver record-count validation
* Gold-table availability validation

All validation checks passed successfully.

12. Final Analytical Outputs

The following CSV files were generated:

* `driver_performance.csv`
* `cancellation_rate.csv`
* `revenue_summary.csv`
* `delay_report.csv`
* `high_demand_locations.csv`

These files contain business-ready analytical results generated from the Gold Layer.

13. Key Results

* Total source drivers: 150
* Total source trips: 150
* Total trip logs: 150
* Active drivers with at least one trip: 99
* Completed trips: 63
* Cancelled trips: 87
* Overall cancellation rate: 58%
* Cities analyzed: 5
* Pickup locations analyzed: 5
* Duplicate primary keys found: 0
* Missing driver references: 0
* Missing trip-log references: 0
* Invalid numeric records: 0

* Records lost during processing: 0

14. Conclusion

This project successfully implemented an end-to-end ride-sharing data engineering pipeline using Python, PySpark, Apache Spark, Spark SQL, Databricks, and Parquet.

The pipeline converted raw operational datasets into clean, validated, integrated, and business-ready analytical outputs.

The project demonstrates practical data engineering skills in:

* Data ingestion
* Schema definition
* Data cleaning
* Data validation
* Dataset integration
* Feature engineering
* Spark SQL
* Window functions
* Aggregation
* Parquet storage
* CSV export
* Data-quality monitoring

The final outputs can support operational monitoring, driver-performance evaluation, cancellation analysis, delay analysis, demand analysis, and revenue-related decision-making.